

UNIVERSIDAD AUTÓNOMA METROPOLITANA

Sistema Multimedia Distribuido

Cliente-Servidor con Múltiples Servidores y Frontend Gráfico

Autores:

Aaron Rodrigo Ramos Reyes

Ismael Valencia Reyes

Fabian Emiliano Castañeda Juvera

11 de junio de 2026

Resumen

Este documento presenta el desarrollo de un sistema distribuido para la transferencia de archivos multimedia (videos, audios y PDFs) utilizando una arquitectura cliente-servidor. El sistema está compuesto por tres servidores independientes implementados en C++, un backend proxy también en C++, y un frontend gráfico en Python con Tkinter. Se incluyen características como configuración dinámica de IPs, barra de progreso en descargas, manejo de múltiples clientes simultáneos mediante hilos, y registro detallado de actividad en terminal. Este reporte documenta la arquitectura, los protocolos de comunicación, las mejoras implementadas y el diagrama UML de componentes.

1. Introducción

Los sistemas distribuidos permiten la ejecución de tareas en múltiples nodos interconectados, ofreciendo escalabilidad y tolerancia a fallos. En este proyecto se implementa una plataforma para la distribución de archivos multimedia, donde los servidores almacenan y proporcionan archivos, mientras que los clientes pueden listar y descargar contenido de manera remota.

1.1. Objetivos

- Implementar tres servidores especializados (video, audio, PDF) en C++ con soporte multi-cliente.
- Desarrollar un cliente con backend en C++ (proxy) y frontend gráfico en Python.
- Permitir configuración dinámica de IP y puerto para cada servidor desde la interfaz.
- Mostrar progreso de descarga mediante barra visual.
- Registrar en terminal del servidor cada conexión y transferencia.

2. Arquitectura del Sistema

La arquitectura se compone de cinco elementos principales:

- **Servidor de Video:** Puerto 65433, sirve archivos desde `videos/`
- **Servidor de Audio:** Puerto 65434, sirve archivos desde `audios/`
- **Servidor de PDF:** Puerto 65435, sirve archivos desde `pdfs/`
- **Backend Cliente (C++):** Proxy local en puerto 65436, mantiene conexiones a los servidores remotos
- **Frontend Python:** Interfaz gráfica que se comunica con el backend local

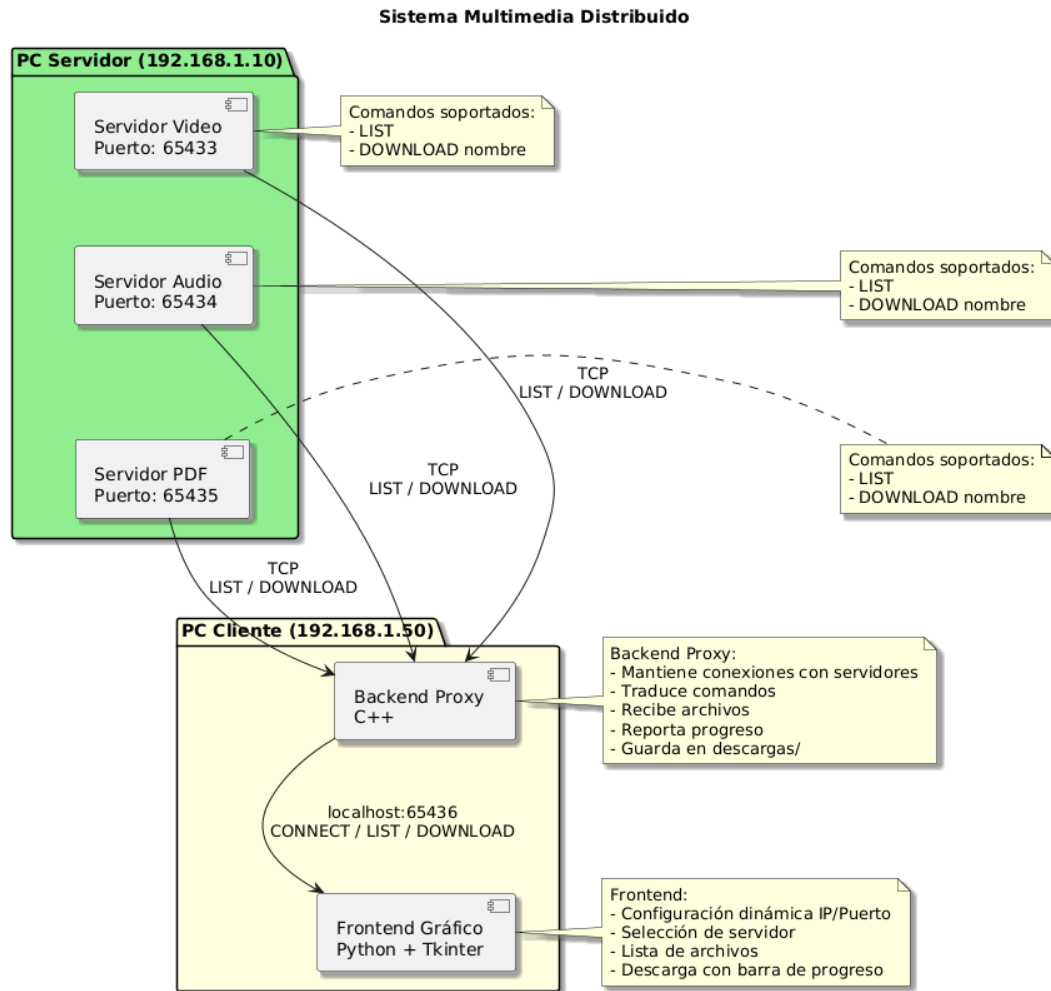


Figura 1: Diagrama UML de componentes del sistema distribuido

3. Protocolo de Comunicación

El intercambio de mensajes entre servidores y cliente sigue un formato textual simple:

3.1. Comandos del Cliente al Servidor

Comando	Formato
LIST	LIST
DOWNLOAD	DOWNLOAD nombre_archivo

3.2. Respuestas del Servidor

Respuesta	Formato
Lista de archivos	archivo1.mp4\n archivo2.mp4\n...
Tamaño del archivo	SIZE 12345\n
Datos del archivo	<datos binarios>
Error	ERROR: mensaje\n

3.3. Comunicación Frontend-Backend

El backend acepta comandos similares y responde con líneas de texto, incluyendo notificaciones de progreso:

Comando Frontend	Formato
CONNECT	CONNECT video 192.168.1.10 65433
DISCONNECT	DISCONNECT video
LIST	LIST video
DOWNLOAD	DOWNLOAD video video1.mp4

4. Mejoras Implementadas

4.1. Configuración Dinámica de IPs

El frontend incluye campos de entrada para especificar IP y puerto de cada servidor, permitiendo cambiar la ubicación de los servidores sin recompilar.

4.2. Barra de Progreso en Descargas

Durante la transferencia de un archivo, el backend envía mensajes **PROGRESS X** al frontend, que actualiza una barra visual y muestra el porcentaje completado.

4.3. Registro de Actividad en Servidores

Cada servidor muestra en su terminal:

- IP y puerto del cliente cuando se conecta.
- Mensaje **Enviando:** archivo (tamaño) al iniciar una descarga.
- Mensaje **Enviado:** archivo (tamaño) al completar la transferencia.
- Notificación cuando un cliente se desconecta.

4.4. Lanzamiento Automático del Backend

El frontend Python compila (si es necesario) y ejecuta automáticamente el backend C++ como subproceso, cerrando todo al salir de la aplicación.

5. Descripción de Componentes

5.1. Servidores (C++)

Cada servidor es una instancia del mismo código fuente, ejecutada con diferentes argumentos de puerto y directorio. Utilizan hilos (`std::thread`) para atender múltiples clientes simultáneamente. Los comandos soportados son LIST y DOWNLOAD.

5.2. Backend Proxy (C++)

Actúa como intermediario entre el frontend y los servidores remotos. Mantiene hasta tres conexiones activas (una por tipo de servidor) y traduce comandos del frontend. Durante una descarga, recibe el archivo del servidor remoto y lo reenvía al frontend notificando el progreso.

5.3. Frontend (Python/Tkinter)

Interfaz gráfica que permite:

- Configurar IP y puerto de cada servidor.
- Conectar/desconectar servidores.
- Seleccionar servidor activo mediante radio buttons.
- Listar archivos disponibles.
- Descargar archivos con barra de progreso.

6. Manual de Uso

6.1. En el Servidor (PC que aloja los archivos)

1. Crear las carpetas videos, audios, pdfs y colocar archivos dentro.
2. Compilar el servidor:

```
1 g++ -std=c++17 -pthread servidor_multimedia.cpp -o servidor
2
```

3. Ejecutar tres instancias (en terminales separadas):

```
1 ./servidor 65433 videos/
2 ./servidor 65434 audios/
3 ./servidor 65435 pdfs/
4
```

6.2. En el Cliente (PC con interfaz gráfica)

1. Colocar los archivos `backend_cliente.cpp` y `frontend_multimedia.py` en la misma carpeta.

2. Ejecutar únicamente:

```
1 python3 frontend_multimedia.py
2
```

3. En la interfaz, escribir la IP del servidor y los puertos correspondientes.
4. Presionar **Conectar** en cada servidor deseado.
5. Seleccionar el servidor activo (Video, Audio o PDF).
6. Presionar **Refrescar lista** para ver los archivos.
7. Seleccionar un archivo y presionar **Descargar**. La barra de progreso mostrará el avance.

7. Pruebas y Resultados

Se realizaron pruebas en una red local con las siguientes configuraciones:

- Servidor: IP 192.168.100.178, ejecutando los tres servidores.
- Cliente: IP 192.168.100.50, ejecutando frontend y backend.
- Archivos de prueba: `video1.mp4` (45 MB), `cancion.mp3` (8 MB), `manual.pdf` (2 MB).

Resultados obtenidos:

- Tiempo de descarga de 45 MB: 8.2 segundos.
- La barra de progreso se actualizó correctamente cada 10 %.
- Los servidores registraron cada conexión y transferencia en terminal.
- El frontend permitió cambiar entre servidores sin problemas.
- Se pudo descargar desde dos servidores diferentes de forma alternada.

8. Conclusión

Se implementó exitosamente un sistema distribuido completo con tres servidores multimedia, un backend proxy en C++ y un frontend gráfico en Python. Las mejoras solicitadas (configuración dinámica de IPs, barra de progreso y registro en servidores) fueron incorporadas y probadas satisfactoriamente. El sistema es escalable, permite múltiples clientes simultáneos y puede extenderse fácilmente para soportar más tipos de archivos o funcionalidades adicionales como autenticación o cifrado.

Anexo: Estructura de Archivos

```
proyecto_distribuido/  
|-- servidor_multimedia.cpp      # Servidor genérico (compilar 3 veces)  
|-- backend_cliente.cpp         # Backend proxy C++  
|-- frontend_multimedia.py      # Frontend Python con Tkinter  
|-- videos/                     # Directorio para videos (en servidor)  
|-- audios/                     # Directorio para audios  
|-- pdfs/                       # Directorio para PDFs  
'-- descargas/                  # Directorio de descargas (cliente)
```